

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME:Software Testing

COURSE CODE: CSA3712

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3

4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3

10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3

17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Solutions:

1. Testing as an Engineering Activity – Banking Application

Testing should be treated as a systematic engineering activity rather than an informal process of randomly executing the application. For the banking application, the testing approach should begin with requirement analysis, where functional and non-functional requirements such as login, account balance, fund transfer, withdrawal, security, and performance are identified. Based on these requirements, systematic test cases should be designed using techniques such as **equivalence partitioning, boundary value analysis, decision table testing, and state transition testing**. Test management and defect tracking tools such as Jira or similar tools can be used to record test cases, execution results, defects, severity, priority, and status. Standard testing practices should include test plans, test case templates, review procedures, traceability matrices, and defined entry and exit criteria. Automation tools can be introduced for repetitive regression tests, while manual exploratory testing can be used for unexpected behavior. For example, in an ATM withdrawal function, test cases should verify valid withdrawals, insufficient balance, invalid PIN, maximum withdrawal limits, and network failures. This engineering-based approach replaces ad-hoc testing with

planned and measurable testing, thereby improving defect detection, test coverage, repeatability, and overall software quality. Therefore, treating testing as an engineering discipline ensures that defects are detected systematically before they affect customers.

2. Structured Testing Process Across SDLC – ATM Withdrawal

A structured testing process should be implemented throughout the Software Development Life Cycle rather than performing testing only after coding is completed. During the **requirement analysis phase**, testers review requirements and identify ambiguities such as withdrawal limits and insufficient-balance conditions. During the **design phase**, test scenarios and test conditions are prepared based on the ATM workflow. During the **implementation phase**, developers perform unit testing for individual functions such as PIN validation and balance calculation. During the integration testing phase, the ATM application is tested with the bank server, account database, and transaction system. System testing then validates the complete withdrawal process from card insertion to cash dispensing and receipt generation. Acceptance testing verifies whether the system satisfies customer and business requirements. For example, an ATM withdrawal test should verify a valid card, correct PIN, sufficient balance, permitted amount, successful cash dispensing, and correct balance update. Negative cases such as incorrect PIN, insufficient balance, invalid amount, and network failure must also be tested. Maintaining the same test process, documentation, standards, and defect reporting practices across SDLC phases reduces inconsistency and prevents defects from moving to later stages. Thus, a structured SDLC testing process improves reliability, reduces rework, and ensures that the ATM performs correctly under normal and exceptional conditions.

3. Errors, Defects, and Failures – Web Application

In software testing, an **error** is a human mistake made during requirements, design, coding, or other development activities. A **defect** is the incorrect condition in the software caused by that error, while a **failure** occurs when the software executes and produces an incorrect or unexpected result. For example, suppose a developer misunderstands the login requirement and writes code that accepts an incorrect password. The developer's misunderstanding is an error, the incorrect password-validation logic present in the program is a defect, and when a user enters an incorrect password and is still allowed to log in, the observed incorrect behavior is a failure. Similarly, if a transaction is expected to transfer ₹5,000 but the application transfers ₹50,000, the programming mistake may be the error, the incorrect calculation logic is the defect, and the incorrect transfer observed during execution is the failure. Understanding these terms is important because testers must report the observable failure and provide sufficient information to help developers locate the underlying defect. A clear defect report should contain the defect ID, module, steps to reproduce, expected result, actual result, severity, priority, environment, screenshots or logs, and status.

Therefore, correctly distinguishing error, defect, and failure improves communication, debugging efficiency, and defect resolution.

4. Defect Clustering and Test Planning

Defect clustering is the testing principle that states that a small number of modules often contain a large proportion of software defects. Therefore, test planning should not distribute testing effort equally across all modules but should give greater attention to high-risk and historically defective areas. For example, in an e-commerce application, the payment, checkout, and order-processing modules may contain more defects because they involve complex business rules and external integrations. Test cases for these modules should therefore be prioritized based on defect history, complexity, business criticality, and risk. Other testing principles such as **exhaustive testing is impossible, early testing, pesticide paradox, absence-of-errors fallacy, and testing shows the presence of defects** should also guide planning. Historical defect repository data can identify modules with repeated failures and help testers create additional regression and negative test cases. For example, if payment processing has repeatedly failed because of invalid card details, timeout conditions, and duplicate transactions, these scenarios should receive higher priority in future releases. Applying defect clustering allows testing resources to be focused where they provide maximum value. Hence, risk-based prioritization and defect history improve testing efficiency, reduce critical failures, and increase the effectiveness of limited testing time.

5. Defect Classification and Defect Repository

A defect repository is a centralized collection of information about defects identified during software development and testing. A company can improve its defect management by classifying defects according to their nature, such as **functional defects, performance defects, usability defects, security defects, interface defects, data defects, and compatibility defects**. Defects can originate from incorrect requirements, poor design, coding mistakes, integration problems, configuration errors, or inadequate testing. For example, if a banking application calculates interest incorrectly, it may be classified as a functional or logic defect. If the application becomes extremely slow when many users log in, it may be classified as a performance defect. Repository information such as defect module, root cause, severity, frequency, release, and resolution can be analyzed to identify recurring patterns. If the same login defect occurs in multiple releases, testers can create permanent regression test cases for that condition and developers can perform root-cause analysis. The repository should include defect ID, description, steps to reproduce, expected and actual results, severity, priority, environment, assigned person, root cause, corrective action, and closure status. Thus, effective use of defect repositories changes testing from merely

fixing individual defects to learning from previous defects and preventing their recurrence, resulting in better quality and reduced maintenance effort.

6. Mobile Banking Application – Fund Transfer Failures

Frequent crashes during fund transfer indicate that the application requires systematic functional, integration, negative, and performance testing. The testing process should first identify the complete transfer workflow, including login, beneficiary selection, amount entry, balance verification, authentication, transaction processing, database update, and confirmation. Testers should then design positive and negative test cases for valid transfers, insufficient balance, invalid beneficiary details, zero or negative amounts, maximum transaction limits, network interruption, duplicate submission, session timeout, and server failure. Boundary value analysis can be used for minimum and maximum transaction amounts, while state transition testing can verify different transaction states such as initiated, processing, successful, failed, and cancelled. Logs, stack traces, crash reports, and database records should be analyzed to identify the root cause. For example, if the application crashes when the network disconnects immediately after the transfer button is pressed, the problem may be caused by improper exception handling or transaction-state management. Automated regression tests should be executed after every major change, and performance and security testing should also be included. This systematic approach helps identify failures earlier and prevents similar crashes from reaching users, thereby improving the reliability and trustworthiness of mobile banking services.

7. Testing and Debugging

Testing and debugging are related but different software activities. **Testing** is performed to identify failures and demonstrate that the software does not behave as expected, whereas **debugging** is the process of locating the root cause of a detected defect and correcting the code. For example, if a banking application displays an incorrect account balance after a transaction, the tester identifies and reports the failure with proper evidence. The developer then performs debugging by examining the transaction logic, database update, and calculation code to locate and fix the defect. Confusing testing with debugging can result in testers attempting to modify source code or developers assuming that successful debugging means complete testing has been performed. A structured approach should therefore separate responsibilities while maintaining collaboration: testers design and execute test cases, report failures, developers investigate and fix defects, and testers perform retesting and regression testing after the fix. The defect should be closed only after the corrected behavior is verified. This separation provides independent evaluation while ensuring effective communication between testers and developers. Therefore, testing discovers problems, while debugging diagnoses and removes their causes; both are essential for achieving high software quality.

8. Cross-Browser Testing – E-Commerce Platform

Cross-browser testing is essential for an e-commerce platform because the same web application may behave differently in Chrome, Edge, Firefox, Safari, and on different operating systems and screen sizes. Defects may originate from browser-specific rendering engines, unsupported JavaScript features, CSS differences, browser versions, responsive-design problems, or third-party integrations. A systematic strategy should first identify the browsers, versions, operating systems, devices, and screen resolutions used by the target customers. Test cases should then cover critical functions such as login, product search, product display, cart, checkout, payment, and order confirmation across the selected browser combinations. Responsive testing should also verify mobile, tablet, and desktop layouts. Tools such as Selenium, Playwright, Cypress, or cloud-based browser testing platforms can help automate repetitive cross-browser tests. For example, if the checkout button works correctly in Chrome but is not displayed properly in Firefox, the defect should be recorded with browser version, operating system, steps to reproduce, expected result, and actual result. Systematic cross-browser testing increases compatibility, reduces customer-facing failures, and ensures consistent user experience. Therefore, combining risk-based browser selection, automation, responsive testing, and manual validation provides effective coverage.

9. Payroll System – Logical Defects

Incorrect salary calculations in a payroll system can be classified primarily as **functional or logic defects**, because the software fails to implement the required salary calculation rules correctly. Such defects may occur because of incorrect formulas, wrong conditions, incorrect tax calculations, improper deductions, data-type errors, or misunderstanding of requirements. For example, if an employee's basic salary is ₹30,000 and the application incorrectly calculates a deduction of ₹10,000 instead of ₹3,000 because of an incorrect formula, the defect is a logic or calculation defect. Understanding defect classes helps testers select appropriate testing techniques. Boundary value analysis can be applied to salary ranges, decision tables can be used for tax and allowance rules, and equivalence partitioning can divide employee categories into valid groups. Testers should also perform unit testing of individual calculation functions, integration testing with employee databases, and regression testing after changes. Peer code reviews and automated unit tests can prevent recurring calculation errors. A strong test strategy should include normal, boundary, exceptional, and negative cases. Therefore, defect classification helps identify the correct testing approach and improves the prevention and detection of payroll calculation errors.

10. Structured Test Case Design

Random testing without proper test case design is inefficient because it may repeatedly test common scenarios while missing important conditions and edge cases. Testing should instead be treated as an engineering activity based on requirements, risks, test objectives, and measurable coverage. Test cases should clearly specify the test case ID, requirement, preconditions, input data, execution steps, expected result, actual result, and status. Techniques such as **equivalence partitioning, boundary value analysis, decision table testing, state transition testing, use-case testing, and exploratory testing** can be used to systematically derive test cases. For example, if an online banking system allows a transfer between ₹100 and ₹50,000, test cases should include values below the minimum, at the minimum, normal values, at the maximum, and above the maximum rather than testing only ₹5,000. Requirements traceability should also be maintained to ensure that every important requirement has corresponding test cases. Automation can be used for repetitive regression tests. This structured approach provides better coverage, avoids unnecessary duplication, identifies boundary defects, and makes test execution measurable and repeatable. Therefore, well-designed test cases are significantly more effective than random testing.

11. Healthcare Application – Data Consistency

Data inconsistency in a healthcare application is a serious quality issue because incorrect patient information can affect medical decisions and patient safety. Such defects may originate from incorrect database operations, inconsistent data formats, synchronization problems, incorrect mapping between modules, concurrency issues, or programming errors. For example, if a patient's blood group is displayed as O+ in the patient-registration module but O– in the medical-record module, the system has a data-integrity defect. Testers should perform integration testing to verify data flow between registration, appointment, laboratory, billing, and medical-record modules. Database validation should confirm that the same patient ID maps to consistent information across tables and modules. Boundary, negative, concurrency, and transaction testing should also be performed. A defect repository should record the affected module, data involved, steps to reproduce, expected and actual values, database evidence, severity, and root cause. Automated data validation and regression tests can verify consistency after every release. Therefore, systematic integration and data-integrity testing, supported by accurate defect tracking, helps maintain reliable healthcare information and prevents potentially serious consequences.

12. Repeated Defects and Defect Repository

Repeated defects indicate that the organization is recording defects but not effectively learning from them. A defect repository should be used not only to track the status of individual defects but also to identify patterns, root causes, and areas requiring additional testing. Each defect should contain information such as

module, defect type, severity, priority, release, environment, root cause, corrective action, and recurrence history. For example, if payment failures occur repeatedly because of timeout handling, repository analysis can reveal that the payment module has a recurring integration problem. Testers can then create specific regression tests for timeout, retry, duplicate payment, and network interruption conditions. Defect trends can also be analyzed according to module, release, severity, and root cause to identify high-risk areas. Root-cause analysis and preventive actions should be performed instead of simply correcting individual defects. The repository should therefore be connected to regression test planning and release reviews. By converting historical defect information into new test scenarios and preventive development practices, organizations can reduce recurring defects, improve release quality, and decrease testing and maintenance costs.

13. Web-Based Learning System – Peak Usage

Failure during peak usage indicates that the learning system was not adequately tested for performance and realistic user loads. Testing gaps may include insufficient load testing, stress testing, scalability testing, concurrency testing, and inadequate coverage of peak-hour usage scenarios. Test planning should identify expected numbers of concurrent students, simultaneous login requests, video access, assignment submissions, examinations, and database transactions. For example, if the system normally supports 1,000 users but an examination causes 10,000 students to log in simultaneously, load testing should reproduce this condition before deployment. Performance metrics such as response time, throughput, CPU utilization, memory usage, database response time, and error rate should be measured. Stress testing can determine the point at which the system becomes unstable, while scalability testing evaluates whether additional resources can handle increased demand. Tools such as JMeter or similar performance-testing tools can be used to simulate concurrent users. Therefore, proper performance test planning and realistic workload scenarios identify bottlenecks before production and improve system reliability, availability, and user experience during peak usage.

14. Early Involvement of Testers in SDLC

Testers should be involved from the early stages of the SDLC because early defect detection is less costly and more effective than finding defects after implementation. During requirement analysis, testers can review requirements for ambiguity, incompleteness, inconsistency, and testability. During design, they can identify potential integration, usability, security, and performance risks and prepare test scenarios in advance. For example, suppose a banking requirement states that a customer can transfer money but does not define the maximum transfer amount or behavior during network failure. A tester involved during requirement analysis can identify these missing conditions before development begins. This prevents

developers from implementing unclear requirements and reduces rework. Testers should participate in requirement reviews, design reviews, sprint planning, acceptance-criteria discussions, and defect-prevention meetings. Collaboration between developers, testers, business analysts, and stakeholders should be continuous rather than limited to the testing phase. This approach supports the principle of **early testing**, improves requirement quality, reduces development costs, and increases overall software quality. Therefore, involving testers early changes testing from defect detection alone to defect prevention.

15. Incomplete Test Coverage

A system that works correctly in some cases but fails in others indicates incomplete test coverage. Complete testing of every possible input is generally impossible, so test cases should be designed systematically to cover important requirements, conditions, risks, and input combinations. For example, an online payment system should not be tested only with successful transactions; it should also be tested with invalid cards, insufficient balance, expired cards, network failures, duplicate clicks, transaction timeouts, maximum amounts, and cancelled payments. Equivalence partitioning can divide inputs into valid and invalid groups, while boundary value analysis can test values at and around limits. Decision tables are useful when several business conditions combine, and state transition testing can verify different transaction states. Requirement traceability matrices can ensure that important requirements are linked to corresponding test cases. Code coverage can provide additional information about executed program paths, although it should not be considered a substitute for requirement-based testing. Improving coverage through risk-based and systematic test design increases the probability of detecting hidden defects and improves system reliability. Therefore, meaningful test coverage is essential for validating software under both normal and exceptional conditions.

16. Defect Reporting in Logistics Application

Poor defect reporting delays identification and resolution because developers may not have enough information to reproduce or understand the problem. A good defect report should clearly distinguish the observed failure from the underlying defect and provide sufficient evidence for investigation. Each report should include a unique defect ID, title, module, environment, build version, preconditions, steps to reproduce, test data, expected result, actual result, severity, priority, screenshots, logs, and status. For example, instead of reporting “delivery tracking is not working,” a tester should report that after entering a valid tracking ID and selecting Track, the expected delivery location should be displayed, but the system shows an incorrect location. The defect should also include the browser/device and relevant logs. Severity indicates the impact of the defect, while priority indicates how urgently it should be fixed. Standardized templates and defect-management tools can ensure consistency across the testing team. Clear reports reduce

communication gaps, speed up debugging, and make defect trends easier to analyze. Therefore, structured defect reporting is an essential part of an effective software testing process.

17. Process-Based Testing

Process-based testing ensures that testing activities are performed consistently according to defined procedures, standards, and quality criteria. If different teams test different modules using different approaches, important scenarios may be missed and results may become difficult to compare. A standardized process should define requirement analysis, test planning, test design, test execution, defect reporting, retesting, regression testing, and test closure. For example, in an e-commerce system, every module such as login, product search, cart, payment, and order tracking should follow a common test case format, defect-reporting procedure, severity classification, and approval process. Checklists, reusable test templates, coding standards, automation frameworks, and defined entry and exit criteria can improve consistency. A requirement traceability matrix can ensure that each requirement is tested across relevant modules. Periodic reviews and test audits can verify whether teams are following the defined process. Standardization does not mean that every module must have identical test cases; rather, it ensures that the underlying testing discipline remains consistent. Therefore, process-based testing improves repeatability, accountability, coverage, communication, and overall software quality.

18. Untested Edge Cases and Boundary Testing

Failures after deployment often occur because testing focuses mainly on normal inputs and ignores boundary and exceptional conditions. **Boundary value analysis** is an effective technique for identifying such defects because errors frequently occur at the edges of input ranges. For example, if an application allows users aged between 18 and 60 to register, test cases should include 17, 18, 19, 59, 60, and 61 rather than testing only an ordinary age such as 30. Similarly, if a banking system allows withdrawals up to ₹50,000, testers should test values such as ₹49,999, ₹50,000, and ₹50,001. Edge cases should also include empty values, maximum-length input, zero values, invalid formats, duplicate requests, network interruption, timeout, and unexpected user actions. Test design should combine boundary value analysis with equivalence partitioning, decision tables, and negative testing. Requirements should explicitly identify limits and exceptional conditions during planning. By deliberately including edge cases in test suites and regression testing, organizations can detect defects that may otherwise appear only in production. Therefore, boundary testing improves reliability by validating the behavior of software at its most vulnerable input conditions.

19. Defect Repository and Documentation

A properly maintained defect repository provides a centralized and traceable record of all software defects and their resolution history. When defects are poorly documented, teams may waste time reproducing problems, duplicate existing reports, or fail to identify recurring defects. Each repository entry should contain a unique defect ID, title, description, affected module, environment, steps to reproduce, expected result, actual result, severity, priority, screenshots or logs, assigned developer, root cause, fix details, retest result, and closure status. For example, if a login failure occurs only after multiple unsuccessful password attempts, the repository should record the exact sequence and environment so that developers can reproduce it. Repository data can be analyzed to identify defect trends by module, type, severity, release, and root cause. This information can then be used to create regression test cases and improve future development practices. A standardized defect-reporting template and regular repository reviews should be adopted to maintain accuracy. Therefore, proper documentation improves traceability, communication, accountability, root-cause analysis, and prevention of recurring defects.

20. Banking Transaction Processing – Redesigning Testing

Repeated transaction failures in a banking system indicate that the existing testing process is not sufficiently systematic or comprehensive. The testing approach should begin with detailed requirement analysis covering account validation, balance verification, transaction limits, authentication, database updates, transaction status, rollback, and failure recovery. Test cases should be designed using equivalence partitioning, boundary value analysis, decision tables, state transition testing, negative testing, and risk-based testing. For example, a fund transfer should be tested with valid and invalid account numbers, insufficient balance, minimum and maximum amounts, duplicate transactions, network interruption, timeout, session expiry, and server failure. Integration testing should verify communication between the application, bank server, authentication service, and database, while system and regression testing should validate the complete workflow after every release. Automation can be used for repetitive transaction and regression scenarios, while performance and security testing should verify reliability under high load and protect sensitive financial information. Defect repository data should also be analyzed to identify modules with repeated failures and prioritize additional testing. A structured approach improves coverage, detects defects earlier, reduces recurring failures, and increases customer confidence. Therefore, systematic and risk-based testing is essential for ensuring reliable and secure banking transaction processing.



Code of Conduct:

I R.Rachitha /192421058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. Rachitha

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand